

SAPIENZA
UNIVERSITÀ DI ROMA

Integer Linear Programming

Volume 2 — Exact methods

The cutting plane algorithm with Gomory fractional cuts, LP-based branch-and-bound and branch-and-cut. Examples worked out step by step and exercises with complete solutions.

Fabio Furini

fabiofurini.github.io

Department of Computer, Control and Management Engineering — Sapienza University of Rome

Online version and downloadable material:
fabiofurini.github.io/integer-linear-programming/
github.com/fabiofurini/integer-linear-programming

Academic year 2026–2027

Contents

III Exact methods **2**

6 The cutting plane algorithm with Gomory fractional cuts **3**
6.1 Gomory fractional cuts 3
6.2 A cutting-plane algorithm based on Gomory fractional cuts 14

7 LP-based branch-and-bound **25**
7.1 Why a branch-and-bound algorithm? 26
7.2 The idea in words and in a picture 27
7.3 Definitions and notation 28
7.4 The algorithm 31
7.5 A short recap of the dual simplex algorithm 34
7.6 Branching-variable selection strategies 35
7.7 Branching-node selection strategies 37
7.8 Worked examples on the car example 39
7.9 Typical mistakes 60
7.10 Extensions 60
7.11 In summary 61

8 Branch-and-cut **62**
8.1 Combining cutting planes and branch and bound 62
8.2 A complete example: the car example 65
8.3 Local cuts: an example 71
8.4 Practical aspects 72

Part III

Exact methods

Chapter 7

LP-based branch-and-bound

Prerequisites

- From the module *Linear Programming*, Chapter 10 «Dual simplex algorithm»: simplex dictionaries, dual feasible bases and dual pivoting operations. A short recap is given in this chapter.
- From this module, the chapter «Integer linear programming»: MIP and ILP problems, feasible and optimal solutions, and the car example.
- From this module, the chapter «LP relaxation and dual bounds»: the LP relaxation, lower bounds from feasible solutions, upper bounds from the LP relaxation, and the optimality gap.

Objectives

At the end of this chapter you will be able to:

- explain how branching operations divide a MIP problem into smaller LP problems, and why no feasible solution is lost;
- apply the three pruning criteria, and decide when the bound of a node can be rounded down;
- execute an LP-based branch-and-bound algorithm by hand, re-optimizing every node with the dual simplex algorithm;
- compute the lower bound, the upper bound and the optimality gap at every iteration;
- compare different branching-variable and branching-node selection strategies on the same instance.

7.1 Why a branch-and-bound algorithm?

- We start from the **car example**. A company produces x_1 sports cars and x_2 family cars per week; the two constraints describe the available labour hours and special materials, and the objective is the weekly profit (in thousands of euros). The ILP problem reads as follows:

$$\begin{array}{rrcrcr} \max & 3\,x_1 & + & 2\,x_2 & & \\ \text{subject to} & 10\,x_1 & + & 4\,x_2 & \leq & 45 \\ & 4\,x_1 & + & 14\,x_2 & \leq & 49 \\ & x_1, & & x_2 & \in & \mathbb{Z}_{\geq 0} \end{array}$$

- The optimal solution of the LP relaxation is fractional:

$$\tilde{x}_1 = \frac{7}{2}, \quad \tilde{x}_2 = \frac{5}{2}, \qquad z(\text{LP}) = \frac{31}{2}.$$

- Rounding it down we get the feasible solution $\bar{\boldsymbol{x}} = (3, 2)$ of value 13. Since the costs are integer, the LP relaxation also gives the upper bound $\lfloor z(\text{LP}) \rfloor = 15$:

$$13 \quad \leq \quad z(\text{ILP}) \quad \leq \quad 15.$$

- The feasible region contains 16 integer points. Enumerating them all is still possible here, but the number of integer points grows exponentially with the number of variables.

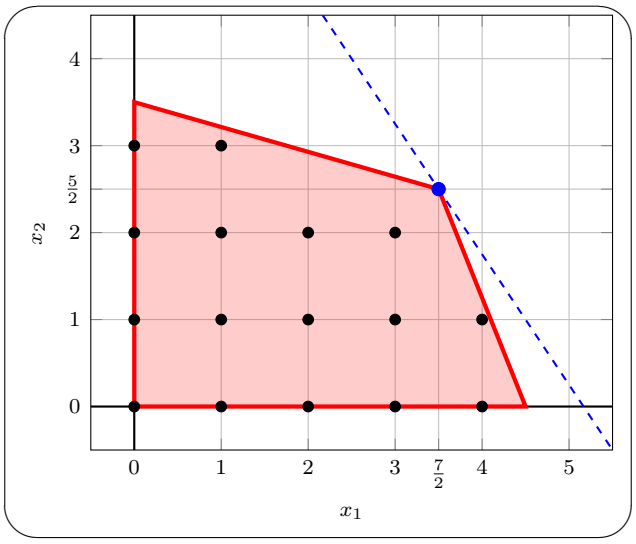


Figure 7.1: The car example: the feasible region of the LP relaxation (red), its 16 integer points (black) and the optimal solution of the LP relaxation (blue), with the objective line through it (dashed).

- **A challenge.** Before reading on, try to answer two questions. Which production plan is optimal? How can we prove that no better plan exists, solving only a few LP problems and without listing all the integer points? The worked examples of this chapter answer both questions.

7.2 The idea in words and in a picture

- The **LP-based branch-and-bound algorithm** (**B&B algorithm**, for short) is an exact method for MIP problems. It is called **LP-based** because it solves LP relaxations to guide the search for an optimal solution.
- In the optimal solution of the LP relaxation of the car example the variable x_1 takes a fractional value between 3 and 4. Every feasible solution of the ILP problem satisfies either $x_1 \leq 3$ or $x_1 \geq 4$, so we can split the problem into two smaller LP problems, one for each side, and remove the strip in between, which contains no integer point:

  $\rightsquigarrow$ **branching operations**.

- Each new LP problem is solved. Its optimal value bounds from above the value of every solution in its region, so it tells us whether the region must be explored further or can be discarded:

  $\rightsquigarrow$ **bounding operations**.

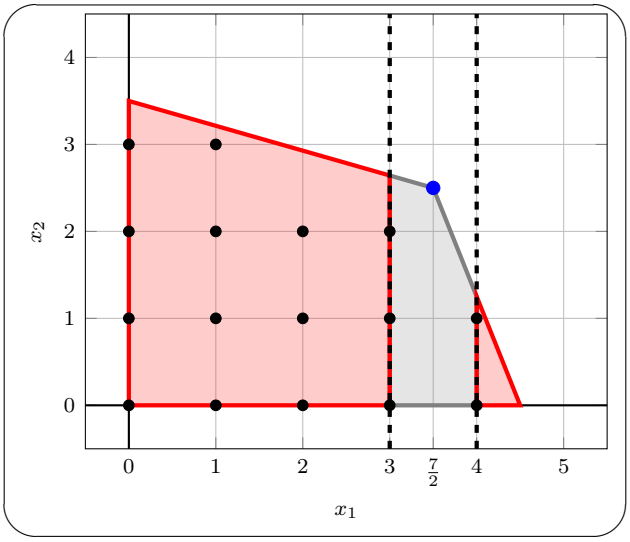


Figure 7.2: Branching on x_1 at the root of the car example: the two red regions are the feasible regions of the children ($x_1 \leq 3$ and $x_1 \geq 4$); the gray strip $3 < x_1 < 4$ is cut off, and it contains no integer point.

A branch-and-bound algorithm explores implicitly all the feasible solutions: it divides the feasible region into smaller regions (branching) and discards every region that cannot contain a better solution than the best one found so far (bounding).

7.3 Definitions and notation

7.3.1 MIP problems in canonical maximization form

- Given n values $c_j \in \mathbb{Q}$, $m \cdot n$ values $a_{ij} \in \mathbb{Q}$, m values $b_i \in \mathbb{Q}$ and an integer $p \in \{1, 2, \dots, n\}$, a **mixed integer linear programming** (MIP) problem in canonical maximization form, with p integer variables and $n - p$ continuous variables, reads as follows:

$$\max \quad \sum_{j=1}^{n} c_j \, x_j \tag{7.1a}$$

$$\text{subject to} \quad \sum_{j=1}^{n} a_{ij} \, x_j \leq b_i, \qquad \forall i \in \{1, 2, \dots, m\}, \tag{7.1b}$$

$$x_j \in \mathbb{Z}_{\geq 0}, \qquad \forall j \in \{1, 2, \dots, p\}, \tag{7.1c}$$

$$x_j \geq 0, \qquad \forall j \in \{p+1, p+2, \dots, n\}. \tag{7.1d}$$

- For $p = n$ all the variables are integer, the constraints (7.1d) disappear, and we obtain the **integer linear programming** (ILP) problem as a special case. The car example is an ILP problem with $n = 2$ and $m = 2$.
- We denote by $z(\text{MIP})$ the optimal value of the problem (7.1), and by $\tilde{\boldsymbol{x}}(\text{MIP})$ an optimal solution. For an ILP problem we write $z(\text{ILP})$ and $\tilde{\boldsymbol{x}}(\text{ILP})$.
- The **LP relaxation** of the problem (7.1), called the LP_1 problem, is obtained by replacing the integrality constraints (7.1c) with the non-negativity constraints $x_j \geq 0$, for all $j \in \{1, 2, \dots, p\}$.

7.3.2 Branch-and-bound tree, incumbent solution and lower bound

The **branching operations** generate a **branch-and-bound tree** containing LP problems. Each LP problem is obtained from the LP relaxation by adding specific **branching constraints**.

- The recursive structure of a B&B algorithm induces the tree. Each **branching node** of the tree corresponds to an LP problem; the root is the LP_1 problem.

Definition 7.1: branching node index

For each branching node k, the associated LP problem is denoted by LP_k. If LP_k has an optimal solution, it is denoted by $\tilde{\boldsymbol{x}}(\text{LP}_k)$ and its optimal value by $z(\text{LP}_k)$. The integer k is called the **branching node index**: it identifies the node uniquely, and the nodes are numbered in the order in which their LP problems are solved.

- A B&B algorithm keeps the best feasible solution of the problem (7.1) found during the search. This solution is called the **incumbent solution** and is denoted by $\bar{\boldsymbol{x}}$; its value is a **lower bound** LB on $z(\text{MIP})$. Before any feasible solution is found we set $\text{LB} = -\infty$.

- The lower bound and the incumbent solution are updated whenever a better feasible solution is found. At termination we have an **algorithmic certificate** that

$$z(\text{MIP}) = \text{LB} \qquad \text{and} \qquad \tilde{\boldsymbol{x}}(\text{MIP}) = \bar{\boldsymbol{x}}. \tag{7.2}$$

7.3.3 Branching operations

- The most common branching operation is the **binary branching**: it acts on a single **branching variable** and produces two child branching nodes, one with an additional **upper bound** and one with an additional **lower bound** on the branching variable.
- More precisely, let $\tilde{\boldsymbol{x}}(\text{LP}_k)$ be an optimal solution of the LP_k problem, and let x_j, with $j \in \{1, 2, \ldots, p\}$, be an integer variable that takes a fractional value $\tilde{x}_j$. The binary branching on x_j generates two child LP problems:
  1. the LP_k problem with the additional **down-branching constraint**

$$x_j \leq \lfloor \tilde{x}_j \rfloor ; \tag{7.3}$$

  2. the LP_k problem with the additional **up-branching constraint**

$$x_j \geq \lfloor \tilde{x}_j \rfloor + 1. \tag{7.4}$$

- The two child nodes are called the **down-branching node** and the **up-branching node**. In both of them $\tilde{\boldsymbol{x}}(\text{LP}_k)$ is no longer feasible.
- Continuous variables are never branching variables: a fractional value of x_j, with $j \in \{p+1, p+2, \ldots, n\}$, does not violate any constraint of the problem (7.1).

Every feasible solution of the MIP problem contained in the region of LP_k satisfies either (7.3) or (7.4), so it is contained in the region of one of the two children. **This ensures the correctness of the B&B algorithm**.

7.3.4 Bounding operations and the bound of a node

- The value $z(\text{LP}_k)$ is an upper bound on the value of every feasible solution of the MIP problem in the region of LP_k. When all the values are integer, this bound can be rounded down.

Observation 7.2: rounding the bound of a node

If all the variables are integer ($p = n$) and all the costs are integer ($c_j \in \mathbb{Z}$ for all $j \in \{1, 2, \ldots, n\}$), then every feasible solution in the region of LP_k has value at most $\lfloor z(\text{LP}_k) \rfloor$. In all the other cases the bound is $z(\text{LP}_k)$.

Proof. If $p = n$ and $c_j \in \mathbb{Z}$ for all j, the value $\sum_{j=1}^{n} c_j \bar{x}_j$ of a feasible solution $\bar{\boldsymbol{x}}$ is a sum of products of integers, hence an integer. Since it is at most $z(\text{LP}_k)$, it is at most the largest integer not exceeding $z(\text{LP}_k)$, that is, $\lfloor z(\text{LP}_k) \rfloor$. □

- Accordingly, we call **bound of the node** k the value

$$\mathrm{UB}(\mathrm{LP}_k) = \begin{cases} \lfloor z(\mathrm{LP}_k) \rfloor & \text{if } p = n \text{ and } c_j \in \mathbb{Z} \text{ for all } j \in \{1, 2, \ldots, n\}, \\ z(\mathrm{LP}_k) & \text{otherwise.} \end{cases} \tag{7.5}$$

- Given a branching node k, no branching operation is performed on it if one of the following **pruning criteria** applies. In these cases the node is said to be **pruned**.

  1. **Pruning by infeasibility**: the LP_k problem is infeasible, so its region contains no feasible solution of the MIP problem.
  2. **Pruning by integrality**: the integer variables have integer values in $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$, so $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ is feasible for the MIP problem, and it is optimal in the region of LP_k. If $z(\mathrm{LP}_k) > \mathrm{LB}$, the lower bound and the incumbent solution are updated.
  3. **Pruning by bound**: the bound of the node does not exceed the lower bound, $\mathrm{UB}(\mathrm{LP}_k) \leq \mathrm{LB}$, so the region of LP_k contains no solution better than the incumbent one.

The criteria are checked in this order: infeasibility, integrality, bound. When both integrality and bound apply, the node is pruned by integrality.

7.3.5 The list of LP problems

- A branching node that is not pruned is inserted into a **list** L of LP problems waiting for a branching operation.
- The order in which the LP problems are extracted from the list determines the **branching-node selection strategy**; the choice of the branching variable determines the **branching-variable selection strategy**. Both are discussed after the algorithm.

7.3.6 Upper bound and optimality gap

- If the list L is not empty, every feasible solution better than the incumbent one lies in the region of some node of the list. Accordingly, the B&B algorithm has the **upper bound**

$$z(\mathrm{MIP}) \quad \leq \quad \mathrm{UB} = \max_{k \in L} \{\mathrm{UB}(\mathrm{LP}_k)\}\,. \tag{7.6}$$

  When the list is empty, the incumbent solution is optimal and UB = LB.

- The upper bound is computed after the solution of the root and updated at the end of every iteration. The algorithm does not need it, but it measures the progress through the **optimality gap**:

$$\text{optimality gap} = \frac{\mathrm{UB} - \mathrm{LB}}{|\mathrm{LB}|}, \qquad \text{for } \mathrm{LB} \neq 0. \tag{7.7}$$

7.3.7 Heuristic initialization

Before the B&B algorithm, a **heuristic algorithm** can be run to find a good feasible solution. The lower bound LB and the incumbent solution $\bar{\boldsymbol{x}}$ are then initialized with this solution.

- A good initial lower bound allows the algorithm to prune nodes by bound earlier, and it can reduce the size of the branch-and-bound tree considerably.
- If the initial lower bound already satisfies $\mathrm{UB}(\mathrm{LP}_1) \leq \mathrm{LB}$, the root is pruned by bound: the heuristic solution is optimal and no branching is needed.

7.4 The algorithm

7.4.1 Pseudocode

Algorithm

Input: An ILP problem in canonical maximization form with integer input data.

Output: An optimal solution to the ILP problem (if one exists) or a certificate of infeasibility/unboundedness.

Step 0: **Initialization**

(a) Relax the integrality constraints and solve the LP_1 problem.

1. if $\tilde{\boldsymbol{x}}(\mathrm{LP}_1)$ is **integer**, then it is an optimal solution and $z(\mathrm{LP}_1)$ is the optimal value of the ILP problem. `STOP`.
2. if the LP_1 problem is **infeasible**, then the ILP problem is infeasible. `STOP`.
3. if the LP_1 problem is **unbounded**, then the ILP problem is either infeasible or unbounded. `STOP`.

(b) Insert the LP_1 problem into a **list of LP problems for branching** (denote L). Set the **lower bound** $\mathrm{LB} \leftarrow -\infty$ (or the value of a heuristic solution), and set a branching node counter $c \leftarrow 1$.

Step 1: **Optimality check**

(a) if the list L is empty and $\mathrm{LB} \neq -\infty$, then LB is the optimal value and the **incumbent solution** $\bar{\boldsymbol{x}}(\mathrm{ILP})$ is an optimal solution to the ILP problem. `STOP`.

(b) if the list L is empty and $\mathrm{LB} = -\infty$, then the ILP problem is infeasible. `STOP`.

Step 2: **Branching-node selection strategy**

Select and remove an LP_k problem from the list L.

Step 3: **Branching-variable selection strategy**

Select a variable x_j with fractional value $\tilde{x}_j$ in $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ and generate two new LP problems:

(a) the LP_{c+1} problem $\rightsquigarrow$ the LP_k problem with the additional **down-branching constraint**: $x_j \leq \lfloor \tilde{x}_j \rfloor$

(b) the LP_{c+2} problem $\rightsquigarrow$ the LP_k problem with the additional **up-branching constraint**: $x_j \geq \lfloor \tilde{x}_j \rfloor + 1$

Execute the **Branching Node Solution Sub-Procedure** for the LP_{c+1} problem and for the LP_{c+2} problem; the second one is skipped if, after the first, $\lfloor z(\text{LP}_k) \rfloor \leq \text{LB}$. Increment the branching node counter $c \leftarrow c + 2$ and go to step 1.

- This is the pseudocode of the **Branching Node Solution Sub-Procedure**.

Algorithm

Input: An LP_k problem associated with a branching node.

Step 1: **Bounding operations**

Solve the LP_k problem.

(a) if the LP_k problem is infeasible, then `STOP`.

(b) if $\lfloor z(\text{LP}_k) \rfloor \leq \text{LB}$, then `STOP`.

(c) if $\tilde{\boldsymbol{x}}(\text{LP}_k)$ is integer, then update the **lower bound** and the **incumbent solution**:

$$\text{LB} \leftarrow z(\text{LP}_k) \quad \text{and} \quad \bar{\boldsymbol{x}}(\text{ILP}) \leftarrow \tilde{\boldsymbol{x}}(\text{LP}_k)$$

Remove from the list L all LP problems with $\lfloor z(\text{LP}) \rfloor \leq \text{LB}$ and `STOP`.

Step 2: **Insertion into the list**

Insert the LP_k problem into the list L and `STOP`.

- The LP problem with the down-branching constraint is called the ***down-branching node***. The LP problem with the up-branching constraint is called the ***up-branching node***. The order in which they are solved determines the order in which the LP problems enter the list L.
- The pseudocode does not say how the LP problems are solved: any method for LP problems can be used.
- If the **dual simplex algorithm** is used, each child is solved starting from the optimal dictionary of its parent: after adding the branching constraint this dictionary is still dual feasible, so in general a few pivot operations are enough (Section 7.5).

7.4.2 The skip rule and the numbering of the nodes

- The bound of a child never exceeds the bound of its parent, since the child has one more constraint. Accordingly, if after the solution of the first child we have $\text{UB}(\text{LP}_k) \leq \text{LB}$, the second child cannot contain a better solution, and it is **not solved**. This is the **skip rule**.
- A skipped child does not receive a node index: the nodes are numbered in the order in which their LP problems are solved, so the indices of the solved nodes are always $1, 2, \ldots, c$.
- The order in which the two children are solved also determines the order in which they enter the list L. In these notes the down-branching node is solved first, unless otherwise stated.

7.4.3 Number of branching nodes

- A **complete** binary branch-and-bound tree with $n \geq 1$ levels has 2^{n-1} leaf branching nodes, and along every root-to-leaf path exactly $n-1$ branching constraints are imposed.

Observation 7.3

The total number of branching nodes in a complete binary branch-and-bound tree with $n \geq 1$ levels is:

$$\sum_{j=0}^{n-1} 2^j = 2^n - 1. \tag{7.8}$$

Proof. By induction on n.

- **Base case.** For $n = 1$ the statement reads $\sum_{j=0}^{0} 2^j = 2^1 - 1$, that is, $1 = 1$, which is true.
- **Inductive step.** Suppose that the statement holds for n; we prove it for $n+1$. We have:

$$\begin{aligned}\sum_{j=0}^{n} 2^j &= \sum_{j=0}^{n-1} 2^j + 2^n = 2^n - 1 + 2^n \quad \text{(by the inductive hypothesis)} \\ &= 2^{n+1} - 1,\end{aligned}$$

  which is the statement for $n+1$.

□

- The total number of nodes is the sum of the first n terms of a geometric progression with ratio $r = 2$:

$$\sum_{j=0}^{n-1} r^j = \begin{cases} \dfrac{r^n - 1}{r - 1} & \text{if } r \neq 1, \\ n & \text{otherwise.} \end{cases} \tag{7.9}$$

- With p binary variables the complete tree has $p+1$ levels and $2^{p+1} - 1$ nodes: the pruning criteria are what makes the algorithm practical.

7.5 A short recap of the dual simplex algorithm

- A child differs from its parent by one constraint. Solving it from scratch would waste the work done for the parent: the B&B algorithm starts instead from the **optimal dictionary of the parent** and re-optimizes it with the **dual simplex algorithm**.
- Let x_j be the branching variable. It is basic in the optimal dictionary of LP_k, since its value is fractional, and its row reads

$$x_j = \bar{b}_j - \sum_{q \in \mathcal{N}} \bar{a}_{jq}\, x_q, \qquad \text{with } \bar{b}_j = \tilde{x}_j.$$

- At depth d of the tree, the branching constraint receives the slack (or surplus) variable x_s, with $s = n + m + d$.

Observation 7.4: the row of the branching constraint

Substituting the row of x_j, the branching constraints become new basic rows of the dictionary:

$$x_j \le \lfloor \tilde{x}_j \rfloor \quad \Longrightarrow \quad x_s = \lfloor \tilde{x}_j \rfloor - x_j = \big(\lfloor \tilde{x}_j \rfloor - \tilde{x}_j\big) + \sum_{q \in \mathcal{N}} \bar{a}_{jq}\, x_q,$$

$$x_j \ge \lfloor \tilde{x}_j \rfloor + 1 \quad \Longrightarrow \quad x_s = x_j - \lfloor \tilde{x}_j \rfloor - 1 = \big(\tilde{x}_j - \lfloor \tilde{x}_j \rfloor - 1\big) - \sum_{q \in \mathcal{N}} \bar{a}_{jq}\, x_q.$$

In both cases the value of x_s is negative, while the reduced costs do not change: the new dictionary is **dual feasible** but not primal feasible.

- This is exactly the starting point of the dual simplex algorithm. In these notes we use the same pivoting rules as in the module *Linear Programming*.

Dual pivoting operation on a dictionary $x_i = \bar{b}_i - \sum_{j \in \mathcal{N}} \bar{a}_{ij}\, x_j$, for $i \in \mathcal{B}$, and $z = \bar{z} + \sum_{j \in \mathcal{N}} \bar{c}_j\, x_j$, with $\bar{c}_j \le 0$ for all $j \in \mathcal{N}$:

a. the **pivot row** is the basic variable with negative value having the smallest index:

$$r = \min \{\ i \in \mathcal{B} : \bar{b}_i < 0\ \};$$

b. if $\bar{a}_{rj} \ge 0$ for all $j \in \mathcal{N}$, the LP problem is **infeasible**;

c. otherwise the **pivot column** x_p attains the minimum ratio (ties: smallest index)

$$\frac{\bar{c}_p}{\bar{a}_{rp}} = s_m = \min \left\{ \frac{\bar{c}_j}{\bar{a}_{rj}} : j \in \mathcal{N},\ \bar{a}_{rj} < 0 \right\};$$

d. after the pivoting operation the objective value is $\bar{z} + s_m\, \bar{b}_r \le \bar{z}$.

The algorithm stops when all the basic variables are non-negative: the dictionary is then optimal.

- In a child node one or two dual pivoting operations are often enough, while the simplex algorithm from the slack basis would need several pivoting operations for every node.

7.6 Branching-variable selection strategies

- The choice of the branching variable has a large impact on the size of the branch-and-bound tree.

Ideally, the branching variable gives the largest decrease of the bounds of the children and leads quickly to feasible solutions. However, the time spent to choose it also counts: an expensive rule may slow down the whole algorithm.

- In all the strategies below, ties are broken by the **smallest index**.

7.6.1 Lexicographic

The **lexicographic** strategy selects the integer variable with a fractional value having the smallest (or the largest) index.

- It requires no computation, but it takes into account neither the decrease of the bounds nor the search for feasible solutions. It is the rule used in the exercises, because it makes the execution easy to reproduce.

7.6.2 Minimum and maximum infeasibility

The **minimum** (respectively, **maximum**) **infeasibility** strategy selects the variable whose fractional value is the closest to (respectively, the farthest from) an integer value.

- Given a branching node k, for each integer variable x_j with a fractional value $\tilde{x}_j$ in $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$, the **fractional value** of the variable is

$$f_j = \min\left\{\tilde{x}_j - \lfloor \tilde{x}_j \rfloor,\ \lceil \tilde{x}_j \rceil - \tilde{x}_j\right\}. \tag{7.10}$$

- Minimum infeasibility tries to reduce the number of fractional variables in the children, to reach feasible solutions faster.

- Maximum infeasibility removes first the variables that contribute more to the infeasibility of $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$. Neither rule looks at the bounds of the children.

Example 7.1: fractional values

At the root node of the car example we have $\tilde{x}_1 = \frac{7}{2}$, $\tilde{x}_2 = \frac{5}{2}$, and the fractional values are:

$$f_1 = \min\left\{\frac{7}{2} - 3,\ 4 - \frac{7}{2}\right\} = \frac{1}{2} \qquad f_2 = \min\left\{\frac{5}{2} - 2,\ 3 - \frac{5}{2}\right\} = \frac{1}{2}$$

The two values are equal: both minimum and maximum infeasibility have a tie, which is broken by the smallest index, so both strategies select x_1.

7.6.3 Strong branching

The **strong branching** strategy selects the variable that gives the largest decrease of the bounds of the children.

- Given a branching node k, for each integer variable x_j with a fractional value, we solve the two children and compute the decreases of the optimal value

  $$\delta_k^d(x_j) = z(\text{LP}_k) - z(\text{down-branching child}), \qquad \delta_k^u(x_j) = z(\text{LP}_k) - z(\text{up-branching child}),$$

  where the decrease is $+\infty$ for an infeasible child.

- The branching variable is x_s, with

  $$s = \arg\max \left\{ \min \left\{ \delta_k^d(x_j), \delta_k^u(x_j) \right\} : j \in \{1, 2, \dots, p\},\ \tilde{x}_j \text{ fractional} \right\}. \tag{7.11}$$

  Both children of the selected variable have a large decrease, so both are more likely to be pruned by bound.

- Strong branching is expensive, since it solves two LP problems for every candidate variable. For this reason it is typically applied only at the top levels of the tree.

Example 7.2: strong branching

At the root node of the car example we have $\tilde{x}_1 = \frac{7}{2}$, $\tilde{x}_2 = \frac{5}{2}$ and $z(\text{LP}_1) = \frac{31}{2}$. For each fractional variable we solve the two child LP problems (with the dual simplex algorithm, from the optimal dictionary of LP_1):

Variable	Down	z	$\delta_1^d(x_j)$	Up	z	$\delta_1^u(x_j)$	$\min\{\delta_1^d, \delta_1^u\}$
x_1	$x_1 \le 3$	$\frac{100}{7}$	$\frac{17}{14}$	$x_1 \ge 4$	$\frac{29}{2}$	1	1
x_2	$x_2 \le 2$	$\frac{151}{10}$	$\frac{2}{5}$	$x_2 \ge 3$	$\frac{45}{4}$	$\frac{17}{4}$	$\frac{2}{5}$

The largest minimum decrease is 1: strong branching selects x_1, the same variable chosen by the smallest-index rule.

7.6.4 Further strategies

- Solvers estimate the decreases δ_k^d and δ_k^u from the history of the search, without solving the children, by means of **pseudocosts**; the same estimates give a branching-node selection rule called **best estimate**. These strategies are a possible extension of this chapter.

7.7 Branching-node selection strategies

- The order in which the nodes of the list L are processed is the **branching-node selection strategy**: it decides where the search goes next.

7.7.1 Best bound

The **best-bound** strategy selects the node of the list L with the largest optimal value $z(\mathrm{LP}_k)$. Ties are broken by the smallest node index.

- The selected node gives the upper bound (7.6): processing it is the only way to decrease UB. The rule tends to explore few nodes, but the list can become long.

7.7.2 LIFO and FIFO

The **Last-In-First-Out** (LIFO) strategy processes the nodes in the reverse order of insertion into the list L. It is equivalent to a **depth-first search** (DFS).

- The list is implemented as a **stack**. The search goes deep into the tree, where the leaves often contain feasible solutions: the goal is to update the lower bound and the incumbent solution quickly. The list stays short.

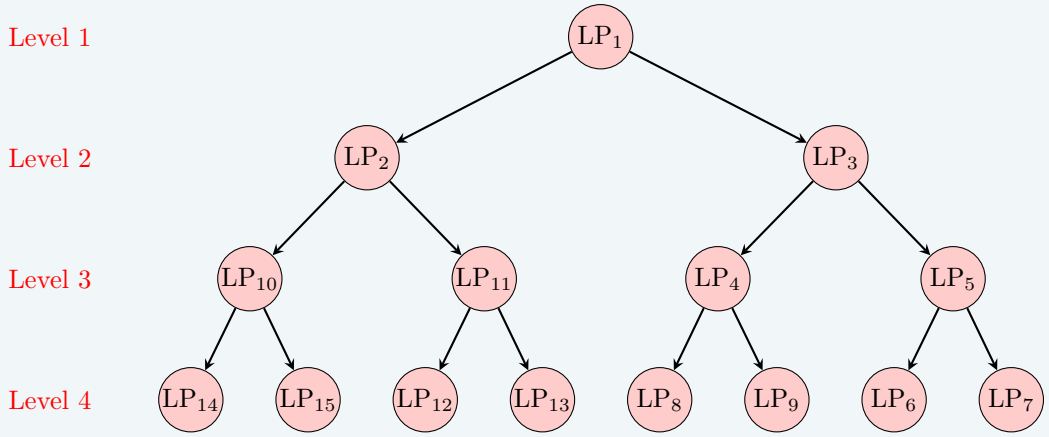
The **First-In-First-Out** (FIFO) strategy processes the nodes in the order of insertion into the list L. It is equivalent to a **breadth-first search** (BFS).

- The list is implemented as a **queue**. The tree is explored level by level: this improves the upper bound, but usually it does not find good lower bounds quickly.

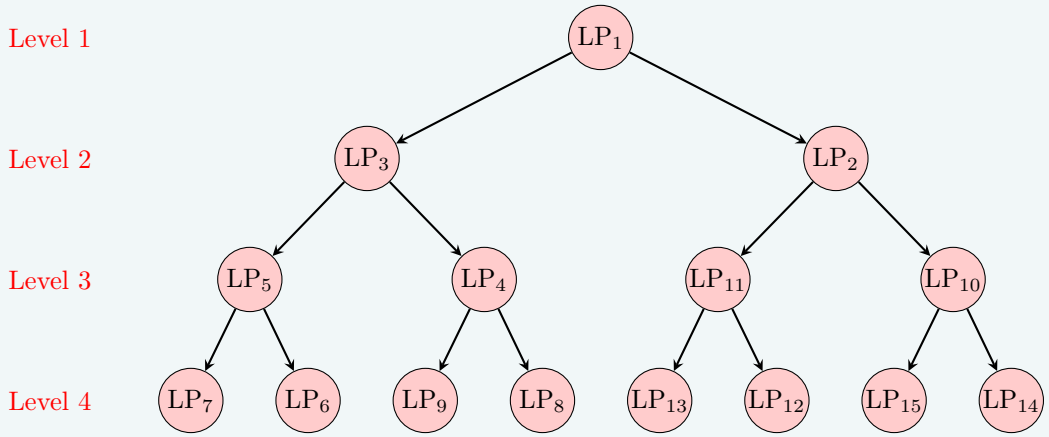
Example 7.3: LIFO strategy

Order in which a complete binary branch-and-bound tree of 4 levels is explored with the LIFO strategy (no node is pruned).

1. If the down-branching node is solved first, we have:



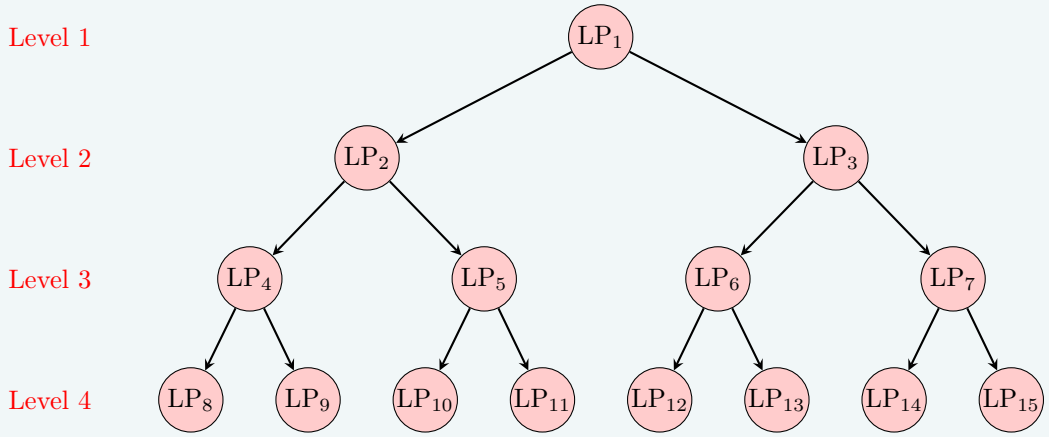
2. If the up-branching node is solved first, we have:



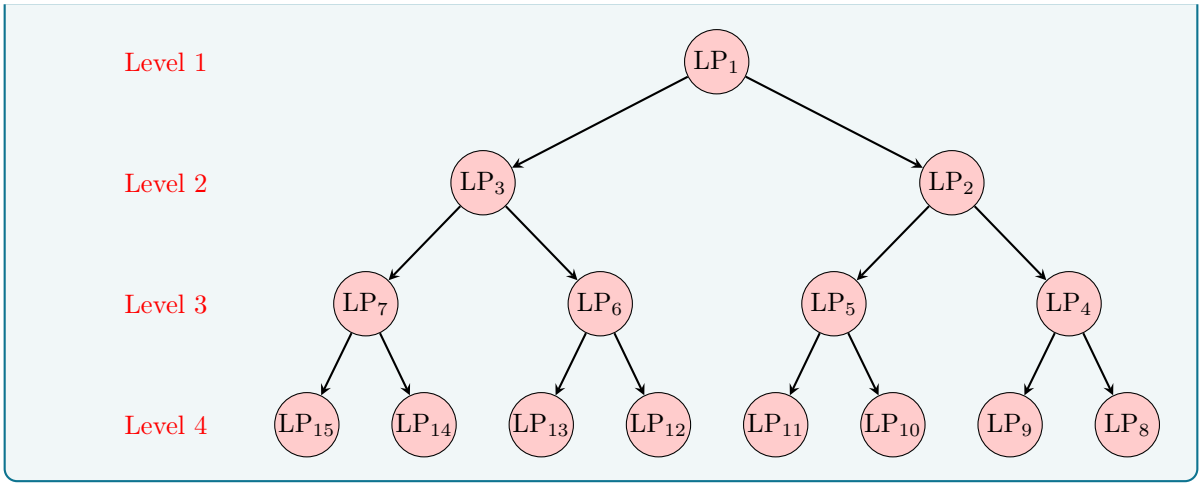
Example 7.4: FIFO strategy

Order in which a complete binary branch-and-bound tree of 4 levels is explored with the FIFO strategy (no node is pruned).

1. If the down-branching node is solved first, we have:



2. If the up-branching node is solved first, we have:



7.8 Worked examples on the car example

- We now solve the car example with two executions of the B&B algorithm, which differ only in the selection strategies. Since all the variables and all the costs are integer, the bound of a node is $\mathrm{UB}(\mathrm{LP}_k) = \lfloor z(\mathrm{LP}_k) \rfloor$.
- In the first execution every node is re-optimized with the dual simplex algorithm, and we report all the dictionaries. For each node a figure shows its feasible region.
- Before reading the solution, try to predict which variable is selected at the root, and which node is extracted at the second iteration.

Example 7.5: execution A: smallest index and LIFO

We execute the B&B algorithm on the car example with:

1. the smallest-index rule (lexicographic) as branching-variable selection strategy;
2. the LIFO rule as branching-node selection strategy;
3. the down-branching node is solved first, then the up-branching node;
4. the second child is not solved when the skip rule applies;
5. the lower bound is initialized to $\mathrm{LB} = -\infty$ (no heuristic algorithm is executed).

- **Initialization**

  The LP_1 problem is the LP relaxation of the car example. The simplex algorithm with Bland's rule, starting from the basis of the slack variables, performs 2 pivoting operations (x_1 enters and x_3 leaves; x_2 enters and x_4 leaves) and ends with the optimal dictionary:

$$\begin{cases} x_1 = \frac{7}{2} - \frac{7}{62} x_3 + \frac{1}{31} x_4 \\ x_2 = \frac{5}{2} + \frac{1}{31} x_3 - \frac{5}{62} x_4 \\ z = \frac{31}{2} - \frac{17}{62} x_3 - \frac{2}{31} x_4 \end{cases}$$

An optimal solution $\tilde{\boldsymbol{x}}(\mathrm{LP}_1)$ is:

$$\tilde{x}_1 = \frac{7}{2}, \quad \tilde{x}_2 = \frac{5}{2}, \qquad z(\mathrm{LP}_1) = \frac{31}{2}.$$

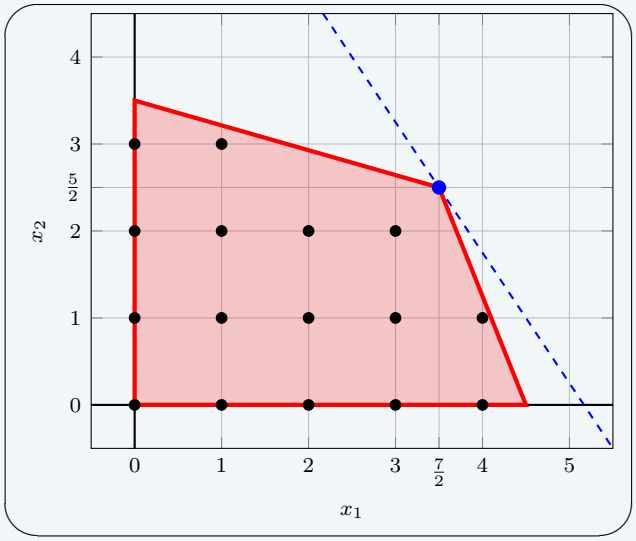


Figure 7.3: Node 1: the feasible region of LP_1, the integer points, the optimal solution of LP_1 (blue) and the objective line through it (dashed).

The solution is fractional and $\lfloor z(\mathrm{LP}_1) \rfloor = \left\lfloor \frac{31}{2} \right\rfloor = 15 > \mathrm{LB} = -\infty$: the LP_1 problem is inserted in the list L.

The list is $L = \{\mathrm{LP}_1\}$, and the upper bound is the largest bound of the nodes in the list:

$$\underbrace{-\infty}_{\mathrm{LB}} \leq z(\mathrm{ILP}) \leq \underbrace{15}_{\mathrm{UB}} \qquad \left(\mathrm{UB} = \max_{k \in L} \mathrm{UB}(\mathrm{LP}_k) = 15\right).$$

- **Iteration 1**

  The list is $L = \{\mathrm{LP}_1\}$: the LP_1 problem is extracted and removed from L.

  In $\tilde{\boldsymbol{x}}(\mathrm{LP}_1)$ we have $\tilde{x}_1 = \frac{7}{2}$ and $\tilde{x}_2 = \frac{5}{2}$, both fractional; by the smallest-index rule the branching variable is x_1. The branching constraints are:

$$x_1 \leq \underbrace{\left\lfloor \frac{7}{2} \right\rfloor}_{=3} \qquad \text{and} \qquad x_1 \geq \underbrace{\left\lfloor \frac{7}{2} \right\rfloor + 1}_{=4}.$$

  1. **Node 2: the down-branching child ($x_1 \leq 3$).**

     The branching constraint $x_1 \leq 3$ is written as $x_1 + x_5 = 3$, with the slack variable $x_5 \geq 0$. Using the row of x_1 in the optimal dictionary of LP_1, we have:

$$x_5 = 3 - x_1 = 3 - \left(\frac{7}{2} - \frac{7}{62}\, x_3 + \frac{1}{31}\, x_4\right) = -\frac{1}{2} + \frac{7}{62}\, x_3 - \frac{1}{31}\, x_4.$$

We add this row to the optimal dictionary of LP_1. The reduced costs do not change, so the new dictionary is **dual feasible**; it is not primal feasible, since $\bar{b}_5 = -\frac{1}{2} < 0$:

$$\begin{cases} x_1 = \frac{7}{2} - \frac{7}{62} x_3 + \frac{1}{31} x_4 \\ x_2 = \frac{5}{2} + \frac{1}{31} x_3 - \frac{5}{62} x_4 \\ x_5 = -\frac{1}{2} + \frac{7}{62} x_3 - \frac{1}{31} x_4 \\ z = \frac{31}{2} - \frac{17}{62} x_3 - \frac{2}{31} x_4 \end{cases}$$

Dual pivoting operation. The pivot row is x_5, the only basic variable with a negative value ($\bar{b}_5 = -\frac{1}{2}$).
The candidates for the pivot column are the non-basic variables with $\bar{a}_{5,j} < 0$:

$$\frac{\bar{c}_3}{\bar{a}_{5,3}} = \frac{-\frac{17}{62}}{-\frac{7}{62}} = \frac{17}{7}$$

The pivot column is x_3, with $s_m = \frac{17}{7}$, and the objective value becomes

$$\bar{z} + s_m \bar{b}_5 = \frac{31}{2} + \frac{17}{7} \cdot \left(-\frac{1}{2}\right) = \frac{100}{7}.$$

After the pivoting operation we obtain the following dictionary, which is primal and dual feasible, hence **optimal**:

$$\begin{cases} x_1 = 3 - x_5 \\ x_2 = \frac{37}{14} - \frac{1}{14} x_4 + \frac{2}{7} x_5 \\ x_3 = \frac{31}{7} + \frac{2}{7} x_4 + \frac{62}{7} x_5 \\ z = \frac{100}{7} - \frac{1}{7} x_4 - \frac{17}{7} x_5 \end{cases}$$

An optimal solution $\tilde{\boldsymbol{x}}(\mathrm{LP}_2)$ is:

$$\tilde{x}_1 = 3, \quad \tilde{x}_2 = \frac{37}{14}, \qquad z(\mathrm{LP}_2) = \frac{100}{7}.$$



Figure 7.4: Node 2: the feasible region of LP_2 (red) inside the region of its parent LP_1 (gray), and the optimal solution of LP_2 (blue) with the objective line through it (dashed).

The solution is **fractional** and

$$\lfloor z(\mathrm{LP}_2) \rfloor = \left\lfloor \frac{100}{7} \right\rfloor = 14 > \mathrm{LB} = -\infty,$$

so the LP_2 problem is **inserted in the list** L.

2. **Node 3: the up-branching child ($x_1 \geq 4$).**
   The branching constraint $x_1 \geq 4$ is written as $x_1 - x_5 = 4$, with the surplus variable $x_5 \geq 0$. Using the row of x_1 in the optimal dictionary of LP_1, we have:

$$x_5 = x_1 - 4 = \left(\frac{7}{2} - \frac{7}{62}\, x_3 + \frac{1}{31}\, x_4\right) - 4 = -\frac{1}{2} - \frac{7}{62}\, x_3 + \frac{1}{31}\, x_4.$$

   We add this row to the optimal dictionary of LP_1. The reduced costs do not change, so the new dictionary is **dual feasible**; it is not primal feasible, since $\bar{b}_5 = -\frac{1}{2} < 0$:

$$\begin{cases} x_1 = \frac{7}{2} - \frac{7}{62}\, x_3 + \frac{1}{31}\, x_4 \\ x_2 = \frac{5}{2} + \frac{1}{31}\, x_3 - \frac{5}{62}\, x_4 \\ x_5 = -\frac{1}{2} - \frac{7}{62}\, x_3 + \frac{1}{31}\, x_4 \\ z = \frac{31}{2} - \frac{17}{62}\, x_3 - \frac{2}{31}\, x_4 \end{cases}$$

   Dual pivoting operation. The pivot row is x_5, the only basic variable with a negative value ($\bar{b}_5 = -\frac{1}{2}$).
   The candidates for the pivot column are the non-basic variables with $\bar{a}_{5,j} < 0$:

$$\frac{\bar{c}_4}{\bar{a}_{5,4}} = \frac{-\frac{2}{31}}{-\frac{1}{31}} = 2$$

The pivot column is x_4, with $s_m = 2$, and the objective value becomes

$$\bar{z} + s_m \bar{b}_5 = \frac{31}{2} + 2 \cdot \left(-\frac{1}{2}\right) = \frac{29}{2}.$$

After the pivoting operation we obtain the following dictionary, which is primal and dual feasible, hence **optimal**:

$$\begin{cases} x_1 = 4 + x_5 \\ x_2 = \frac{5}{4} - \frac{1}{4} x_3 - \frac{5}{2} x_5 \\ x_4 = \frac{31}{2} + \frac{7}{2} x_3 + 31 x_5 \\ z = \frac{29}{2} - \frac{1}{2} x_3 - 2 x_5 \end{cases}$$

An optimal solution $\tilde{\boldsymbol{x}}(\mathrm{LP}_3)$ is:

$$\tilde{x}_1 = 4, \quad \tilde{x}_2 = \frac{5}{4}, \qquad z(\mathrm{LP}_3) = \frac{29}{2}.$$



Figure 7.5: Node 3: the feasible region of LP_3 (red) inside the region of its parent LP_1 (gray), and the optimal solution of LP_3 (blue) with the objective line through it (dashed).

The solution is **fractional** and

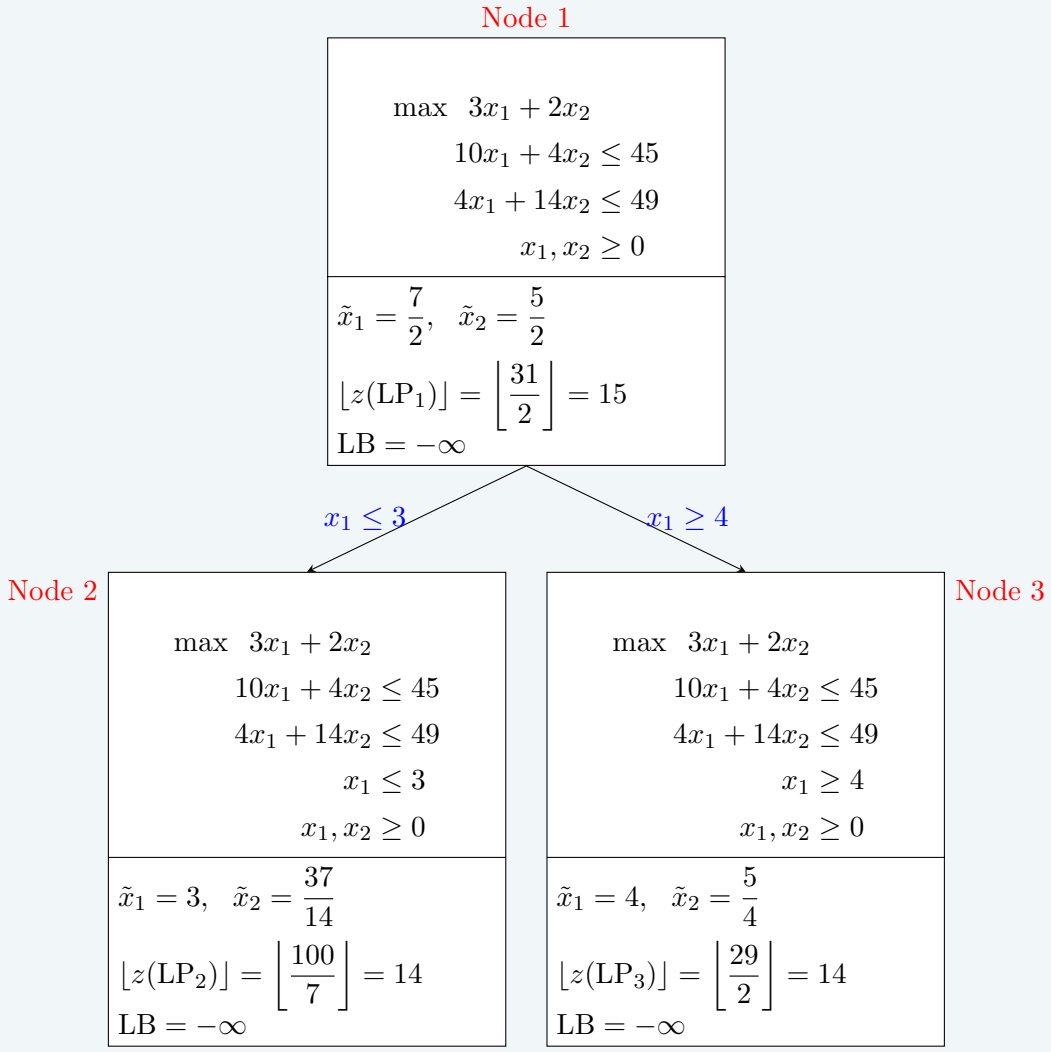
$$\lfloor z(\mathrm{LP}_3) \rfloor = \left\lfloor \frac{29}{2} \right\rfloor = 14 > \mathrm{LB} = -\infty,$$

so the LP_3 problem is **inserted in the list** L.

The list is $L = \{\text{LP}_2,\ \text{LP}_3\}$, and the upper bound is the largest bound of the nodes in the list:

$$\underbrace{-\infty}_{\text{LB}} \ \leq\ z(\text{ILP})\ \leq\ \underbrace{14}_{\text{UB}} \qquad \left(\text{UB} = \max_{k\in L} \text{UB}(\text{LP}_k) = 14\right).$$

This is the portion of the branch-and-bound tree built in the first iteration, with the LP problems of the branching nodes:



- **Iteration 2**

  The list is $L = \{\text{LP}_2,\ \text{LP}_3\}$; following the LIFO rule, LP_3 (the last problem inserted) is extracted and removed from L.

  In $\tilde{\boldsymbol{x}}(\text{LP}_3)$ the only integer variable with a fractional value is x_2, which is the branching variable. The branching constraints are:

$$x_2 \leq \underbrace{\left\lfloor \frac{5}{4} \right\rfloor}_{=1} \qquad \text{and} \qquad x_2 \geq \underbrace{\left\lfloor \frac{5}{4} \right\rfloor + 1}_{=2}.$$

1. **Node 4: the down-branching child ($x_2 \leq 1$).**
   The branching constraint $x_2 \leq 1$ is written as $x_2 + x_6 = 1$, with the slack variable $x_6 \geq 0$. Using the row of x_2 in the optimal dictionary of LP_3, we have:

$$x_6 = 1 - x_2 = 1 - \left(\frac{5}{4} - \frac{1}{4}\,x_3 - \frac{5}{2}\,x_5\right) = -\frac{1}{4} + \frac{1}{4}\,x_3 + \frac{5}{2}\,x_5.$$

   We add this row to the optimal dictionary of LP_3. The reduced costs do not change, so the new dictionary is **dual feasible**; it is not primal feasible, since $\bar{b}_6 = -\frac{1}{4} < 0$:

$$\begin{cases} x_1 = 4 + x_5 \\ x_2 = \frac{5}{4} - \frac{1}{4}\,x_3 - \frac{5}{2}\,x_5 \\ x_4 = \frac{31}{2} + \frac{7}{2}\,x_3 + 31\,x_5 \\ x_6 = -\frac{1}{4} + \frac{1}{4}\,x_3 + \frac{5}{2}\,x_5 \\ z = \frac{29}{2} - \frac{1}{2}\,x_3 - 2\,x_5 \end{cases}$$

   Dual pivoting operation. The pivot row is x_6, the only basic variable with a negative value ($\bar{b}_6 = -\frac{1}{4}$).
   The candidates for the pivot column are the non-basic variables with $\bar{a}_{6,j} < 0$:

$$\frac{\bar{c}_3}{\bar{a}_{6,3}} = \frac{-\frac{1}{2}}{-\frac{1}{4}} = 2, \quad \frac{\bar{c}_5}{\bar{a}_{6,5}} = \frac{-2}{-\frac{5}{2}} = \frac{4}{5}$$

$$s_m = \min\left\{2, \frac{4}{5}\right\} = \frac{4}{5}.$$

   The pivot column is x_5, with $s_m = \frac{4}{5}$, and the objective value becomes

$$\bar{z} + s_m\,\bar{b}_6 = \frac{29}{2} + \frac{4}{5} \cdot \left(-\frac{1}{4}\right) = \frac{143}{10}.$$

   After the pivoting operation we obtain the following dictionary, which is primal and dual feasible, hence **optimal**:

$$\begin{cases} x_1 = \frac{41}{10} - \frac{1}{10}\,x_3 + \frac{2}{5}\,x_6 \\ x_2 = 1 - x_6 \\ x_4 = \frac{93}{5} + \frac{2}{5}\,x_3 + \frac{62}{5}\,x_6 \\ x_5 = \frac{1}{10} - \frac{1}{10}\,x_3 + \frac{2}{5}\,x_6 \\ z = \frac{143}{10} - \frac{3}{10}\,x_3 - \frac{4}{5}\,x_6 \end{cases}$$

An optimal solution $\tilde{\boldsymbol{x}}(\mathrm{LP}_4)$ is:

$$\tilde{x}_1 = \frac{41}{10}, \quad \tilde{x}_2 = 1, \qquad z(\mathrm{LP}_4) = \frac{143}{10}.$$

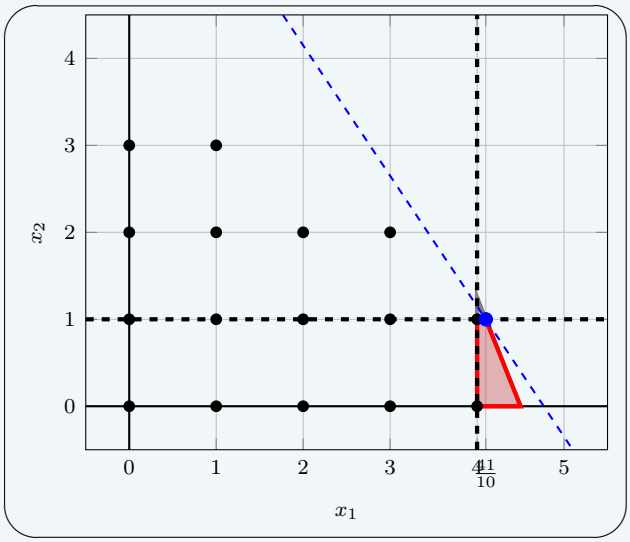


Figure 7.6: Node 4: the feasible region of LP_4 (red) inside the region of its parent LP_3 (gray), and the optimal solution of LP_4 (blue) with the objective line through it (dashed).

The solution is **fractional** and

$$\lfloor z(\mathrm{LP}_4) \rfloor = \left\lfloor \frac{143}{10} \right\rfloor = 14 > \mathrm{LB} = -\infty,$$

so the LP_4 problem is **inserted in the list** L.

2. **Node 5: the up-branching child ($x_2 \geq 2$).**
   The branching constraint $x_2 \geq 2$ is written as $x_2 - x_6 = 2$, with the surplus variable $x_6 \geq 0$. Using the row of x_2 in the optimal dictionary of LP_3, we have:

$$x_6 = x_2 - 2 = \left(\frac{5}{4} - \frac{1}{4}x_3 - \frac{5}{2}x_5\right) - 2 = -\frac{3}{4} - \frac{1}{4}x_3 - \frac{5}{2}x_5.$$

   We add this row to the optimal dictionary of LP_3. The reduced costs do not change, so the new dictionary is **dual feasible**; it is not primal feasible, since $\bar{b}_6 = -\frac{3}{4} < 0$:

$$\begin{cases} x_1 = 4 + x_5 \\ x_2 = \frac{5}{4} - \frac{1}{4}x_3 - \frac{5}{2}x_5 \\ x_4 = \frac{31}{2} + \frac{7}{2}x_3 + 31\,x_5 \\ x_6 = -\frac{3}{4} - \frac{1}{4}x_3 - \frac{5}{2}x_5 \\ z = \frac{29}{2} - \frac{1}{2}x_3 - 2\,x_5 \end{cases}$$

Dual pivoting operation. The pivot row is x_6, the only basic variable with a negative value ($\bar{b}_6 = -\frac{3}{4}$).
All the coefficients of the pivot row are non-negative ($\bar{a}_{6,3} = \frac{1}{4}$, $\bar{a}_{6,5} = \frac{5}{2}$), that is, the row reads

$$x_6 = -\frac{3}{4} - \frac{1}{4}\,x_3 - \frac{5}{2}\,x_5 \leq -\frac{3}{4} < 0$$

for every non-negative value of the non-basic variables. By the dual simplex algorithm, the LP_5 problem is **infeasible**.

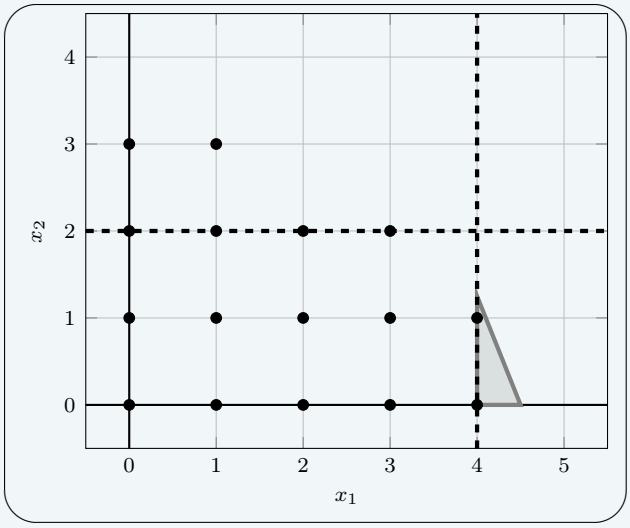


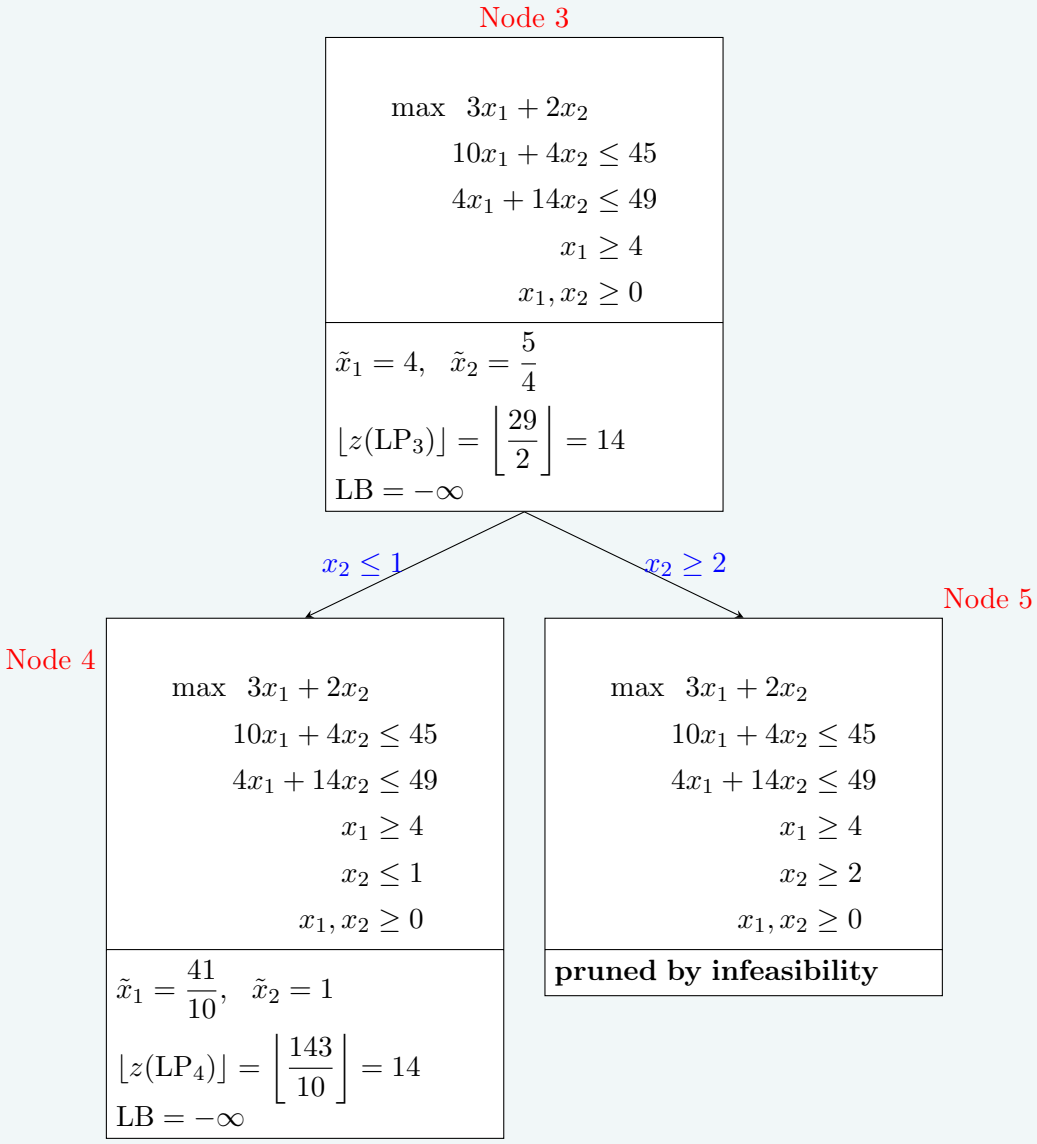
Figure 7.7: Node 5: the constraint $x_2 \geq 2$ leaves no point of the region of its parent LP_3 (gray): LP_5 is infeasible.

Accordingly, Node 5 is pruned by infeasibility.

The list is $L = \{\mathrm{LP}_2,\ \mathrm{LP}_4\}$, and the upper bound is the largest bound of the nodes in the list:

$$\underbrace{-\infty}_{\mathrm{LB}} \leq z(\mathrm{ILP}) \leq \underbrace{14}_{\mathrm{UB}} \qquad \left(\mathrm{UB} = \max_{k \in L} \mathrm{UB}(\mathrm{LP}_k) = 14\right).$$

This is the portion of the branch-and-bound tree built in the second iteration, with the LP problems of the branching nodes:



- **Iteration 3**

  The list is $L = \{\mathrm{LP}_2,\ \mathrm{LP}_4\}$; following the LIFO rule, LP_4 (the last problem inserted) is extracted and removed from L.

  In $\tilde{\boldsymbol{x}}(\mathrm{LP}_4)$ the only integer variable with a fractional value is x_1, which is the branching variable. The branching constraints are:

$$x_1 \le \underbrace{\left\lfloor \frac{41}{10} \right\rfloor}_{=4} \qquad \text{and} \qquad x_1 \ge \underbrace{\left\lfloor \frac{41}{10} \right\rfloor + 1}_{=5}.$$

  1. **Node 6: the down-branching child ($x_1 \le 4$).**

     The branching constraint $x_1 \le 4$ is written as $x_1 + x_7 = 4$, with the slack variable $x_7 \ge 0$. Using the row of x_1 in the optimal dictionary of LP_4, we have:

$$x_7 = 4 - x_1 = 4 - \left(\frac{41}{10} - \frac{1}{10}\,x_3 + \frac{2}{5}\,x_6\right) = -\frac{1}{10} + \frac{1}{10}\,x_3 - \frac{2}{5}\,x_6.$$

We add this row to the optimal dictionary of LP_4. The reduced costs do not change, so the new dictionary is **dual feasible**; it is not primal feasible, since $\bar{b}_7 = -\frac{1}{10} < 0$:

$$\begin{cases} x_1 = \frac{41}{10} - \frac{1}{10}\,x_3 + \frac{2}{5}\,x_6 \\ x_2 = 1 - x_6 \\ x_4 = \frac{93}{5} + \frac{2}{5}\,x_3 + \frac{62}{5}\,x_6 \\ x_5 = \frac{1}{10} - \frac{1}{10}\,x_3 + \frac{2}{5}\,x_6 \\ x_7 = -\frac{1}{10} + \frac{1}{10}\,x_3 - \frac{2}{5}\,x_6 \\ z = \frac{143}{10} - \frac{3}{10}\,x_3 - \frac{4}{5}\,x_6 \end{cases}$$

Dual pivoting operation. The pivot row is x_7, the only basic variable with a negative value ($\bar{b}_7 = -\frac{1}{10}$).
The candidates for the pivot column are the non-basic variables with $\bar{a}_{7,j} < 0$:

$$\frac{\bar{c}_3}{\bar{a}_{7,3}} = \frac{-\frac{3}{10}}{-\frac{1}{10}} = 3$$

The pivot column is x_3, with $s_m = 3$, and the objective value becomes

$$\bar{z} + s_m\,\bar{b}_7 = \frac{143}{10} + 3 \cdot \left(-\frac{1}{10}\right) = 14.$$

After the pivoting operation we obtain the following dictionary, which is primal and dual feasible, hence **optimal**:

$$\begin{cases} x_1 = 4 - x_7 \\ x_2 = 1 - x_6 \\ x_3 = 1 + 4\,x_6 + 10\,x_7 \\ x_4 = 19 + 14\,x_6 + 4\,x_7 \\ x_5 = 0 - x_7 \\ z = 14 - 2\,x_6 - 3\,x_7 \end{cases}$$

An optimal solution $\tilde{\boldsymbol{x}}(\mathrm{LP}_6)$ is:

$$\tilde{x}_1 = 4, \quad \tilde{x}_2 = 1, \qquad z(\mathrm{LP}_6) = 14.$$

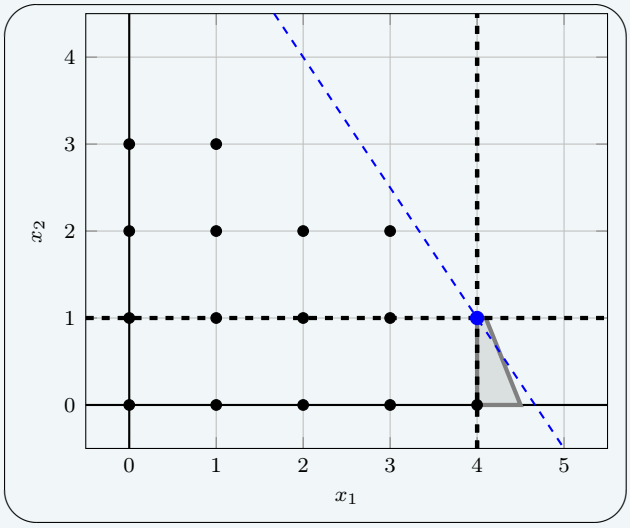


Figure 7.8: Node 6: the feasible region of LP_6 (red) inside the region of its parent LP_4 (gray), and the optimal solution of LP_6 (blue) with the objective line through it (dashed).

The solution $\tilde{\boldsymbol{x}}(\mathrm{LP}_6)$ is **integer**.
Since $z(\mathrm{LP}_6) = 14 > \mathrm{LB} = -\infty$, the lower bound and the incumbent solution are updated:

$$\mathrm{LB} \leftarrow 14, \qquad \bar{\boldsymbol{x}} \leftarrow (4, 1).$$

Node 6 is pruned by integrality.
Moreover, LP_2 is removed from the list L (pruned by bound), since now

$$\lfloor z(\mathrm{LP}_2) \rfloor = \left\lfloor \frac{100}{7} \right\rfloor = 14 \leq \mathrm{LB} = 14.$$

2. **Skip rule.** After Node 6 the lower bound satisfies

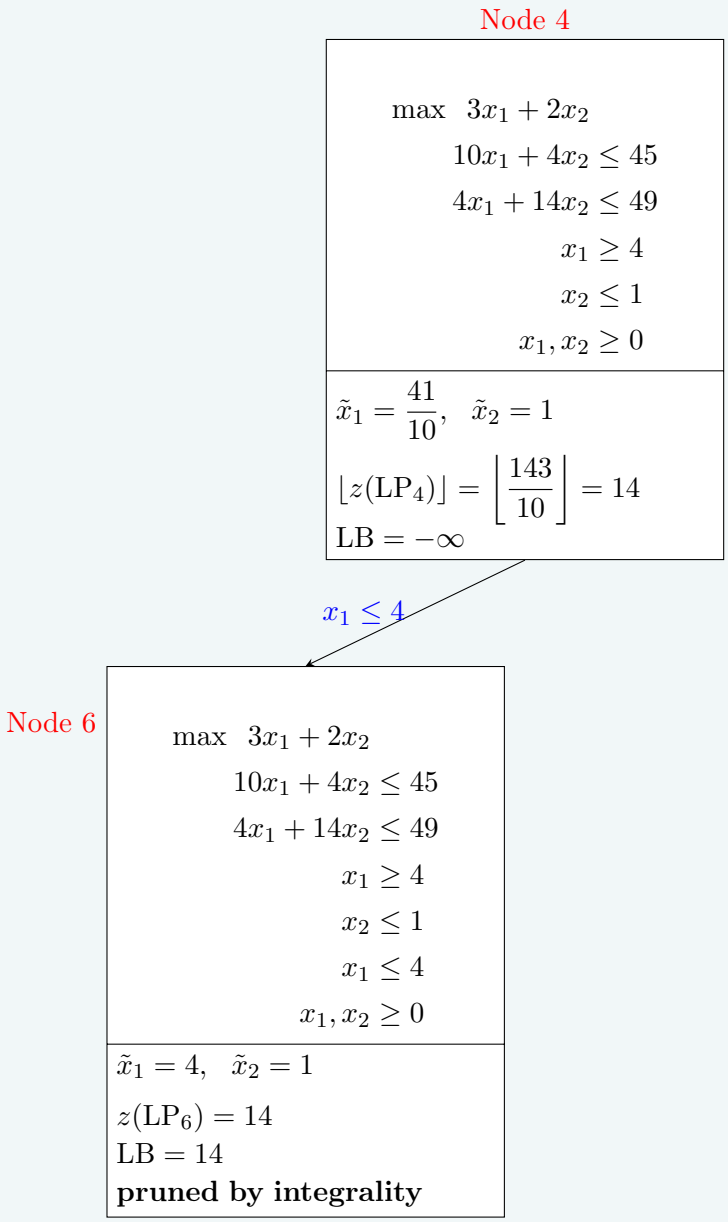
$$\lfloor z(\mathrm{LP}_4) \rfloor = \left\lfloor \frac{143}{10} \right\rfloor = 14 \leq \mathrm{LB} = 14,$$

so no solution in the region of LP_4 can improve the incumbent solution: the up-branching child ($x_1 \geq 5$) is **not solved** and it does not receive a node index.

The list is empty, so the upper bound is equal to the lower bound:

$$\underbrace{14}_{\mathrm{LB}} \leq z(\mathrm{ILP}) \leq \underbrace{14}_{\mathrm{UB}} \qquad (\mathrm{UB} = \mathrm{LB} = 14)\,.$$

This is the portion of the branch-and-bound tree built in the third iteration, with the LP problems of the branching nodes:

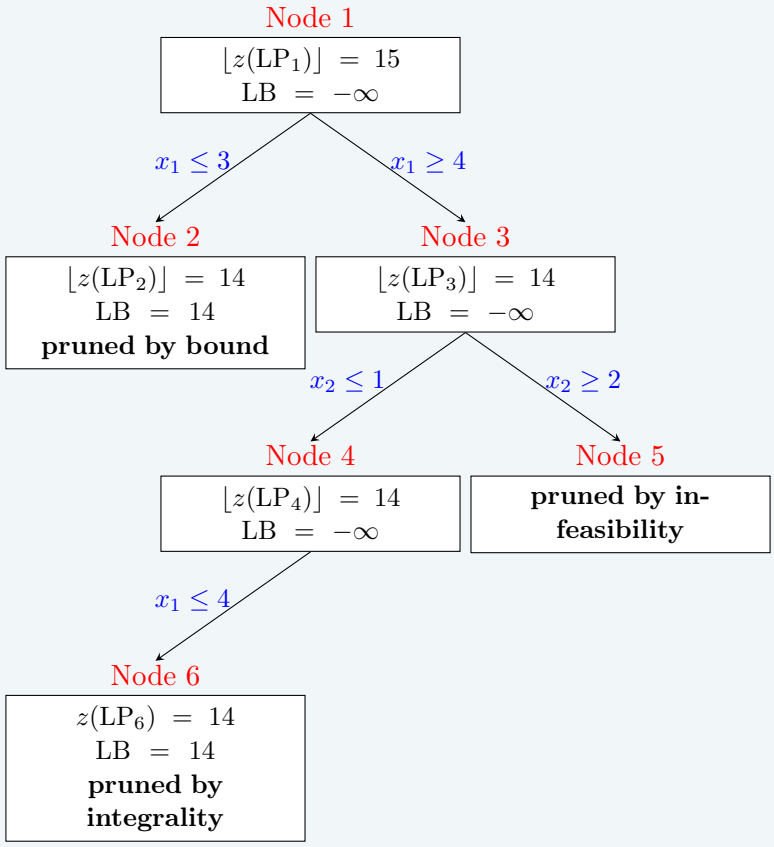


- **Termination**

  The list L is empty: $\text{LB} = \text{UB} = 14$ and the incumbent solution is optimal:

$$\tilde{x}_1 = 4, \quad \tilde{x}_2 = 1, \qquad z(\text{ILP}) = 14.$$

This is the final branch-and-bound tree. For each branching node it shows the bound of the node, the lower bound LB when the node was solved (or removed from the list) and the pruning criterion.



The same execution in a table:

Node	Parent	It.	Branching	$\tilde{\boldsymbol{x}}(\text{LP}_k)$	$z(\text{LP}_k)$	LB	Outcome
1	–	0	–	$(\frac{7}{2}, \frac{5}{2})$	$\frac{31}{2}$	$-\infty$	branched on x_1
2	1	1	$x_1 \leq 3$	$(3, \frac{37}{14})$	$\frac{100}{7}$	14	pruned by bound (removed from L)
3	1	1	$x_1 \geq 4$	$(4, \frac{5}{4})$	$\frac{29}{2}$	$-\infty$	branched on x_2
4	3	2	$x_2 \leq 1$	$(\frac{41}{10}, 1)$	$\frac{143}{10}$	$-\infty$	branched on x_1
5	3	2	$x_2 \geq 2$	–	–	$-\infty$	pruned by infeasibility
6	4	3	$x_1 \leq 4$	$(4, 1)$	14	14	pruned by integrality

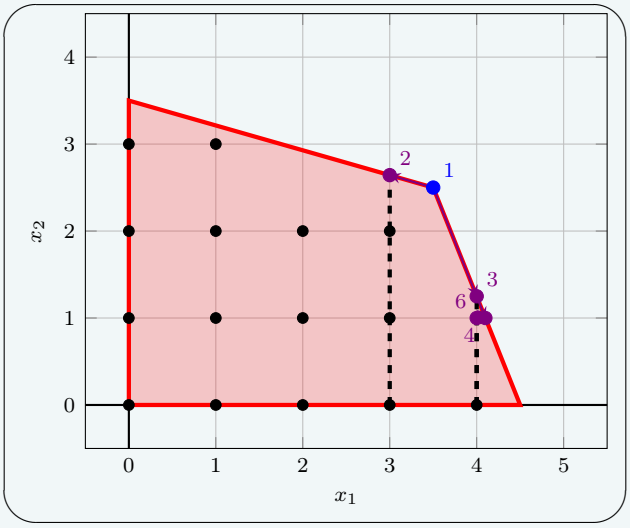


Figure 7.9: Execution A in the plane: the branching constraints (black, dashed) inside the regions of the parent nodes, and the optimal solutions of the solved nodes (violet), from LP_1 (blue).

Example 7.6: execution B: largest index and best bound

We execute again the B&B algorithm on the car example, now with:

1. the largest-index rule (lexicographic) as branching-variable selection strategy;
2. the best-bound rule as branching-node selection strategy (ties: smallest node index);
3. the down-branching node is solved first, then the up-branching node;
4. the second child is not solved when the skip rule applies;
5. the lower bound is initialized to $\mathrm{LB} = -\infty$ (no heuristic algorithm is executed).

Each child is re-optimized with the dual simplex algorithm from the optimal dictionary of its parent, exactly as in Example 7.5; here we report only the results.

- **Initialization**

  As before, $\tilde{\boldsymbol{x}}(\mathrm{LP}_1) = (\frac{7}{2}, \frac{5}{2})$ with $z(\mathrm{LP}_1) = \frac{31}{2}$, and LP_1 is inserted in the list L.

  The list is $L = \{\mathrm{LP}_1\}$, and the upper bound is the largest bound of the nodes in the list:

$$\underbrace{-\infty}_{\mathrm{LB}} \;\leq\; z(\mathrm{ILP}) \;\leq\; \underbrace{15}_{\mathrm{UB}} \qquad \left(\mathrm{UB} = \max_{k \in L} \mathrm{UB}(\mathrm{LP}_k) = 15\right).$$

- **Iteration 1**

  The list is $L = \{\mathrm{LP}_1\}$: the LP_1 problem is extracted and removed from L.

In $\tilde{\boldsymbol{x}}(\mathrm{LP}_1)$ we have $\tilde{x}_1 = \frac{7}{2}$ and $\tilde{x}_2 = \frac{5}{2}$, both fractional; by the largest-index rule the branching variable is x_2. The branching constraints are:

$$x_2 \leq \underbrace{\left\lfloor \frac{5}{2} \right\rfloor}_{=2} \qquad \text{and} \qquad x_2 \geq \underbrace{\left\lfloor \frac{5}{2} \right\rfloor + 1}_{=3}.$$

- Node 2 ($x_2 \leq 2$): $\tilde{\boldsymbol{x}}(\mathrm{LP}_2) = (\frac{37}{10}, 2)$ with $z(\mathrm{LP}_2) = \frac{151}{10}$. The solution is **fractional** and
$$\lfloor z(\mathrm{LP}_2) \rfloor = \left\lfloor \frac{151}{10} \right\rfloor = 15 > \mathrm{LB} = -\infty,$$
so the LP_2 problem is **inserted in the list** L.
- Node 3 ($x_2 \geq 3$): $\tilde{\boldsymbol{x}}(\mathrm{LP}_3) = (\frac{7}{4}, 3)$ with $z(\mathrm{LP}_3) = \frac{45}{4}$. The solution is **fractional** and
$$\lfloor z(\mathrm{LP}_3) \rfloor = \left\lfloor \frac{45}{4} \right\rfloor = 11 > \mathrm{LB} = -\infty,$$
so the LP_3 problem is **inserted in the list** L.

The list is $L = \{\mathrm{LP}_2,\ \mathrm{LP}_3\}$, and the upper bound is the largest bound of the nodes in the list:

$$\underbrace{-\infty}_{\mathrm{LB}} \leq z(\mathrm{ILP}) \leq \underbrace{15}_{\mathrm{UB}} \qquad \left(\mathrm{UB} = \max_{k \in L} \mathrm{UB}(\mathrm{LP}_k) = 15\right).$$

- **Iteration 2**

The list is $L = \{\mathrm{LP}_2,\ \mathrm{LP}_3\}$; following the best-bound rule, LP_2 is extracted and removed from L, since it has the largest optimal value:

$$z(\mathrm{LP}_2) = \frac{151}{10}, \quad z(\mathrm{LP}_3) = \frac{45}{4}.$$

In $\tilde{\boldsymbol{x}}(\mathrm{LP}_2)$ the only integer variable with a fractional value is x_1, which is the branching variable. The branching constraints are:

$$x_1 \leq \underbrace{\left\lfloor \frac{37}{10} \right\rfloor}_{=3} \qquad \text{and} \qquad x_1 \geq \underbrace{\left\lfloor \frac{37}{10} \right\rfloor + 1}_{=4}.$$

- Node 4 ($x_1 \leq 3$): $\tilde{\boldsymbol{x}}(\mathrm{LP}_4) = (3, 2)$ with $z(\mathrm{LP}_4) = 13$. The solution $\tilde{\boldsymbol{x}}(\mathrm{LP}_4)$ is **integer**. Since $z(\mathrm{LP}_4) = 13 > \mathrm{LB} = -\infty$, the lower bound and the incumbent solution are updated:
$$\mathrm{LB} \leftarrow 13, \qquad \bar{\boldsymbol{x}} \leftarrow (3, 2).$$
Node 4 is pruned by integrality. Moreover, LP_3 is removed from the list L (pruned by bound), since now
$$\lfloor z(\mathrm{LP}_3) \rfloor = \left\lfloor \frac{45}{4} \right\rfloor = 11 \leq \mathrm{LB} = 13.$$

– Node 5 ($x_1 \geq 4$): $\tilde{\boldsymbol{x}}(\mathrm{LP}_5) = (4, \frac{5}{4})$ with $z(\mathrm{LP}_5) = \frac{29}{2}$. The solution is **fractional** and

$$\lfloor z(\mathrm{LP}_5) \rfloor = \left\lfloor \frac{29}{2} \right\rfloor = 14 > \mathrm{LB} = 13,$$

so the LP_5 problem is **inserted in the list** L.

The list is $L = \{\mathrm{LP}_5\}$, and the upper bound is the largest bound of the nodes in the list:

$$\underbrace{13}_{\mathrm{LB}} \leq z(\mathrm{ILP}) \leq \underbrace{14}_{\mathrm{UB}} \qquad \left(\mathrm{UB} = \max_{k \in L} \mathrm{UB}(\mathrm{LP}_k) = 14\right).$$

- **Iteration 3**

The list is $L = \{\mathrm{LP}_5\}$: the LP_5 problem is extracted and removed from L.

In $\tilde{\boldsymbol{x}}(\mathrm{LP}_5)$ the only integer variable with a fractional value is x_2, which is the branching variable. The branching constraints are:

$$x_2 \leq \underbrace{\left\lfloor \frac{5}{4} \right\rfloor}_{=1} \qquad \text{and} \qquad x_2 \geq \underbrace{\left\lfloor \frac{5}{4} \right\rfloor + 1}_{=2}.$$

– Node 6 ($x_2 \leq 1$): $\tilde{\boldsymbol{x}}(\mathrm{LP}_6) = (\frac{41}{10}, 1)$ with $z(\mathrm{LP}_6) = \frac{143}{10}$. The solution is **fractional** and

$$\lfloor z(\mathrm{LP}_6) \rfloor = \left\lfloor \frac{143}{10} \right\rfloor = 14 > \mathrm{LB} = 13,$$

so the LP_6 problem is **inserted in the list** L.

– Node 7 ($x_2 \geq 2$): the LP_7 problem is infeasible: pruned by infeasibility.

The list is $L = \{\mathrm{LP}_6\}$, and the upper bound is the largest bound of the nodes in the list:

$$\underbrace{13}_{\mathrm{LB}} \leq z(\mathrm{ILP}) \leq \underbrace{14}_{\mathrm{UB}} \qquad \left(\mathrm{UB} = \max_{k \in L} \mathrm{UB}(\mathrm{LP}_k) = 14\right).$$

- **Iteration 4**

The list is $L = \{\mathrm{LP}_6\}$: the LP_6 problem is extracted and removed from L.

In $\tilde{\boldsymbol{x}}(\mathrm{LP}_6)$ the only integer variable with a fractional value is x_1, which is the branching variable. The branching constraints are:

$$x_1 \leq \underbrace{\left\lfloor \frac{41}{10} \right\rfloor}_{=4} \qquad \text{and} \qquad x_1 \geq \underbrace{\left\lfloor \frac{41}{10} \right\rfloor + 1}_{=5}.$$

– Node 8 ($x_1 \leq 4$): $\tilde{\boldsymbol{x}}(\mathrm{LP}_8) = (4, 1)$ with $z(\mathrm{LP}_8) = 14$. The solution $\tilde{\boldsymbol{x}}(\mathrm{LP}_8)$ is **integer**. Since $z(\mathrm{LP}_8) = 14 > \mathrm{LB} = 13$, the lower bound and the incumbent solution are updated:

$$\mathrm{LB} \leftarrow 14, \qquad \bar{\boldsymbol{x}} \leftarrow (4, 1).$$

Node 8 is pruned by integrality.

– **Skip rule.** After Node 8 the lower bound satisfies

$$\lfloor z(\mathrm{LP}_6) \rfloor = \left\lfloor \frac{143}{10} \right\rfloor = 14 \leq \mathrm{LB} = 14,$$

so no solution in the region of LP_6 can improve the incumbent solution: the up-branching child ($x_1 \geq 5$) is **not solved** and it does not receive a node index.

The list is empty, so the upper bound is equal to the lower bound:

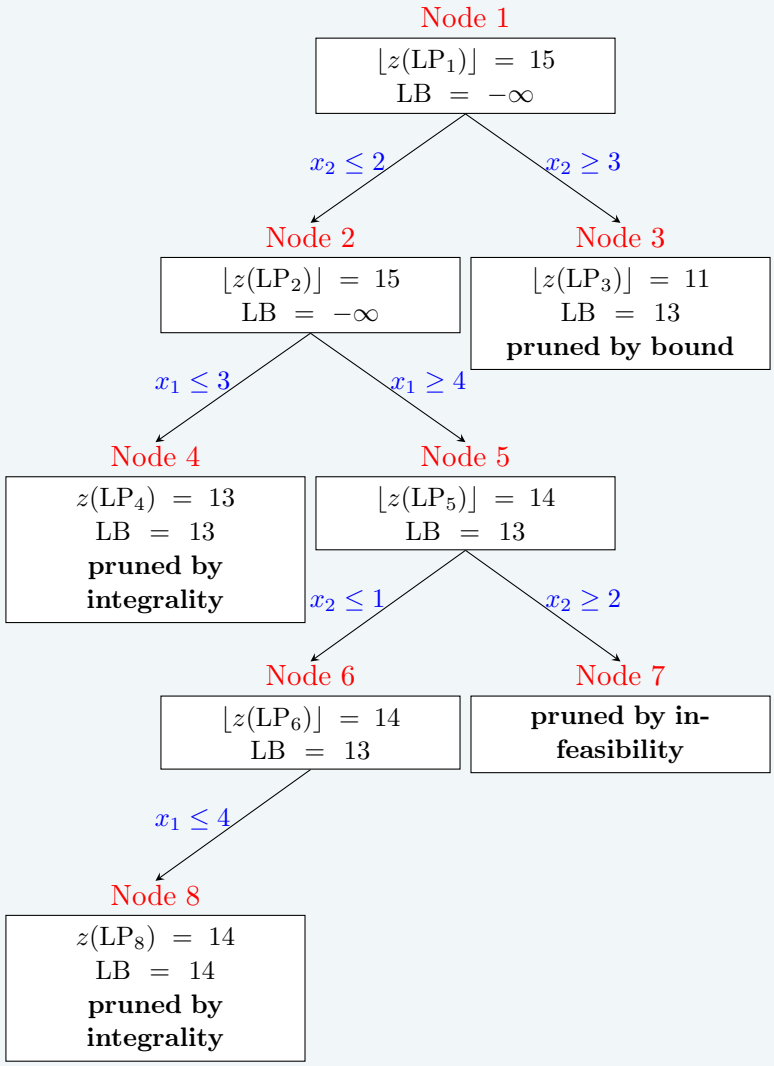
$$\underbrace{14}_{\mathrm{LB}} \leq z(\mathrm{ILP}) \leq \underbrace{14}_{\mathrm{UB}} \qquad (\mathrm{UB} = \mathrm{LB} = 14)\,.$$

- **Termination**

  The list L is empty: $\mathrm{LB} = \mathrm{UB} = 14$ and the incumbent solution is optimal:

$$\tilde{x}_1 = 4, \quad \tilde{x}_2 = 1, \qquad z(\mathrm{ILP}) = 14.$$

This is the final branch-and-bound tree:



Node	Parent	It.	Branching	$\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$	$z(\mathrm{LP}_k)$	LB	Outcome
1	–	0	–	$(\frac{7}{2}, \frac{5}{2})$	$\frac{31}{2}$	$-\infty$	branched on x_2
2	1	1	$x_2 \leq 2$	$(\frac{37}{10}, 2)$	$\frac{151}{10}$	$-\infty$	branched on x_1
3	1	1	$x_2 \geq 3$	$(\frac{7}{4}, 3)$	$\frac{45}{4}$	13	pruned by bound (removed from L)
4	2	2	$x_1 \leq 3$	$(3, 2)$	13	13	pruned by integrality
5	2	2	$x_1 \geq 4$	$(4, \frac{5}{4})$	$\frac{29}{2}$	13	branched on x_2
6	5	3	$x_2 \leq 1$	$(\frac{41}{10}, 1)$	$\frac{143}{10}$	13	branched on x_1
7	5	3	$x_2 \geq 2$	–	–	13	pruned by infeasibility
8	6	4	$x_1 \leq 4$	$(4, 1)$	14	14	pruned by integrality

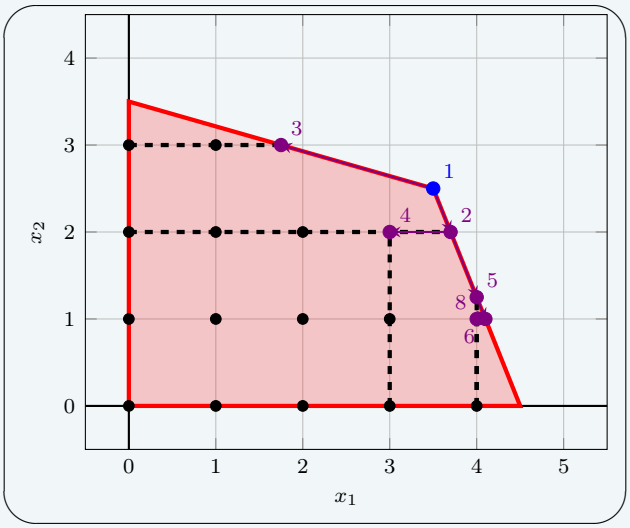


Figure 7.10: Execution B in the plane: branching constraints and optimal solutions of the solved nodes.

- Execution A solves 6 LP problems, execution B solves 8. Both certify the same optimal solution $\tilde{x}_1 = 4, \tilde{x}_2 = 1$ of value 14, which is unique.
- Execution B finds a first incumbent solution of value 13 earlier, but branching first on x_2 leads to a larger tree on this instance.

Example 7.7: heuristic initialization

We execute the B&B algorithm of Example 7.5 (smallest index, LIFO), starting from a heuristic solution of the car example.

- The heuristic that produces only sports cars fixes x_1 as large as possible and $x_2 = 0$: it gives $\bar{\boldsymbol{x}} = (4, 0)$ of value 12. Starting execution A with LB $= 12$, the algorithm solves 6 LP problems, as many as without the heuristic: this lower bound is too weak to prune any node earlier.
- The heuristic that fixes first x_1 and then x_2 as large as possible gives $\bar{\boldsymbol{x}} = (4, 1)$ of

value 14. Starting with LB $= 14$, the root is not pruned, since $\lfloor z(\mathrm{LP}_1) \rfloor = 15 > 14$, but both children are pruned by bound and the algorithm stops after 3 LP problems:

Node	Parent	It.	Branching	$\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$	$z(\mathrm{LP}_k)$	LB	Outcome
1	–	0	–	$(\frac{7}{2}, \frac{5}{2})$	$\frac{31}{2}$	14	branched on x_1
2	1	1	$x_1 \leq 3$	$(3, \frac{37}{14})$	$\frac{100}{7}$	14	pruned by bound
3	1	1	$x_1 \geq 4$	$(4, \frac{5}{4})$	$\frac{29}{2}$	14	pruned by bound

The incumbent solution given by the heuristic is optimal, and the branch-and-bound algorithm **certifies** its optimality.

Example 7.8: a MIP problem

We now allow the number of family cars to be fractional (for example, a weekly average): the variable x_2 becomes **continuous**, and we obtain a MIP problem with $n = 2$, $p = 1$ and $m = 2$:

$$\begin{array}{rrcrcr} \max & 3\,x_1 & + & 2\,x_2 & & \\ \text{subject to} & 10\,x_1 & + & 4\,x_2 & \leq & 45 \\ & 4\,x_1 & + & 14\,x_2 & \leq & 49 \\ & x_1 & & & \in & \mathbb{Z}_{\geq 0} \\ & & & x_2 & \geq & 0 \end{array}$$

- The LP relaxation is the same as before. Only x_1 can be a branching variable, and a node is pruned by integrality as soon as x_1 is integer, whatever the value of x_2.
- Since x_2 is continuous, the value of a feasible solution need not be integer: the bound of a node is $\mathrm{UB}(\mathrm{LP}_k) = z(\mathrm{LP}_k)$, without rounding.
- The LP problems of the nodes are those of the first nodes of Example 7.5, with the same dictionaries.

We use the rules of Example 7.5:

- **Initialization**

  As before, $\tilde{\boldsymbol{x}}(\mathrm{LP}_1) = (\frac{7}{2}, \frac{5}{2})$ with $z(\mathrm{LP}_1) = \frac{31}{2}$, and LP_1 is inserted in the list L.

  The list is $L = \{\mathrm{LP}_1\}$, and the upper bound is the largest bound of the nodes in the list:

  $$\underbrace{-\infty}_{\mathrm{LB}} \leq z(\mathrm{MIP}) \leq \underbrace{\frac{31}{2}}_{\mathrm{UB}} \qquad \left(\mathrm{UB} = \max_{k \in L} \mathrm{UB}(\mathrm{LP}_k) = \frac{31}{2}\right).$$

- **Iteration 1**

  The list is $L = \{\mathrm{LP}_1\}$: the LP_1 problem is extracted and removed from L.

In $\tilde{\boldsymbol{x}}(\mathrm{LP}_1)$ the only integer variable with a fractional value is x_1, which is the branching variable. The branching constraints are:

$$x_1 \leq \underbrace{\left\lfloor \frac{7}{2} \right\rfloor}_{=3} \qquad \text{and} \qquad x_1 \geq \underbrace{\left\lfloor \frac{7}{2} \right\rfloor + 1}_{=4}.$$

- Node 2 ($x_1 \leq 3$): $\tilde{\boldsymbol{x}}(\mathrm{LP}_2) = (3, \frac{37}{14})$ with $z(\mathrm{LP}_2) = \frac{100}{7}$. The integer variable has an integer value in $\tilde{\boldsymbol{x}}(\mathrm{LP}_2)$: the solution is **feasible for the MIP problem**. Since $z(\mathrm{LP}_2) = \frac{100}{7} > \mathrm{LB} = -\infty$, the lower bound and the incumbent solution are updated:
$$\mathrm{LB} \leftarrow \frac{100}{7}, \qquad \bar{\boldsymbol{x}} \leftarrow (3, \frac{37}{14}).$$
Node 2 is pruned by integrality.
- Node 3 ($x_1 \geq 4$): $\tilde{\boldsymbol{x}}(\mathrm{LP}_3) = (4, \frac{5}{4})$ with $z(\mathrm{LP}_3) = \frac{29}{2}$. The integer variable has an integer value in $\tilde{\boldsymbol{x}}(\mathrm{LP}_3)$: the solution is **feasible for the MIP problem**. Since $z(\mathrm{LP}_3) = \frac{29}{2} > \mathrm{LB} = \frac{100}{7}$, the lower bound and the incumbent solution are updated:
$$\mathrm{LB} \leftarrow \frac{29}{2}, \qquad \bar{\boldsymbol{x}} \leftarrow (4, \frac{5}{4}).$$
Node 3 is pruned by integrality.

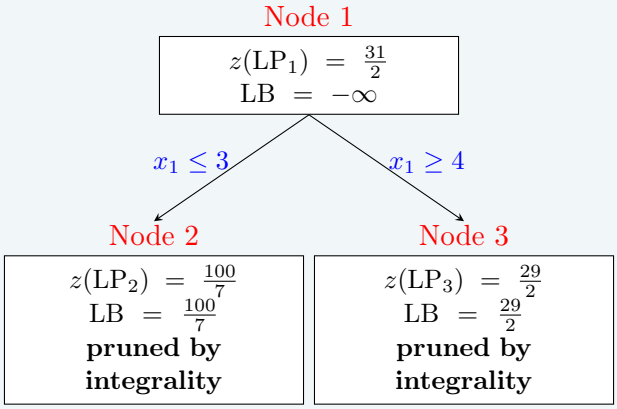
The list is empty, so the upper bound is equal to the lower bound:

$$\underbrace{\frac{29}{2}}_{\mathrm{LB}} \leq z(\mathrm{MIP}) \leq \underbrace{\frac{29}{2}}_{\mathrm{UB}} \qquad \left(\mathrm{UB} = \mathrm{LB} = \frac{29}{2}\right).$$

- **Termination**

  The list L is empty: $\mathrm{LB} = \mathrm{UB} = \frac{29}{2}$ and the incumbent solution is optimal:

$$\tilde{x}_1 = 4, \quad \tilde{x}_2 = \frac{5}{4}, \qquad z(\mathrm{MIP}) = \frac{29}{2}.$$



The optimal solution of the MIP problem is $\tilde{x}_1 = 4, \tilde{x}_2 = \frac{5}{4}$ of value $\frac{29}{2}$, larger than $z(\mathrm{ILP}) = 14$: allowing x_2 to take fractional values enlarges the feasible region.

7.9 Typical mistakes

Warning

- **Rounding the bound of a MIP problem.** The bound $\lfloor z(\text{LP}_k) \rfloor$ is valid only if all the variables and all the costs are integer (Observation 7.2). In the mixed example, when Node 3 is solved we have LB $= \frac{100}{7}$ and $z(\text{LP}_3) = \frac{29}{2}$. The correct test gives $z(\text{LP}_3) = \frac{29}{2} >$ LB, and Node 3 contains the optimal solution. The rounded test would give $\lfloor z(\text{LP}_3) \rfloor = 14 \leq \text{LB} = \frac{100}{7}$: the node would be pruned by mistake, and the algorithm would return the value $\frac{100}{7}$ instead of $\frac{29}{2}$.
- **Rounding the LP solution.** Rounding an optimal solution of the LP relaxation gives at best a feasible solution, in general not an optimal one. In the car example, rounding down $\tilde{\boldsymbol{x}}(\text{LP}) = (\frac{7}{2}, \frac{5}{2})$ gives $(3, 2)$ of value 13, while $z(\text{ILP}) = 14$ at $(4, 1)$.
- **Branching on the wrong variable.** The branching variable must be an integer variable with a fractional value: a continuous variable, or an integer variable with an integer value, does not cut off $\tilde{\boldsymbol{x}}(\text{LP}_k)$.
- **Taking the upper bound from the last node.** The upper bound is the largest bound among the nodes of the list L, not the bound of the last node solved.
- **Pruning by bound only with a strict inequality.** A node with $\text{UB}(\text{LP}_k) = \text{LB}$ is pruned: its region can contain solutions as good as the incumbent one, but not better.
- **Numbering the skipped children.** A child that is not solved does not receive a node index.
- **Solving every node from scratch.** The answer is correct, but much slower: each child starts from the optimal dictionary of its parent, which is dual feasible.

7.10 Extensions

- The LP problems of the nodes can also be solved with the **bounded-variable simplex algorithm**, which handles the branching constraints as bounds on the variables instead of new constraints. This extension is not covered in this course.

7.11 In summary

In summary

- A B&B algorithm divides the feasible region with branching constraints on integer variables with fractional values, and it never loses a feasible solution.
- Every node is pruned by infeasibility, by integrality or by bound, or it waits in the list; the bound of a node is $\lfloor z(\text{LP}_k) \rfloor$ only if all the variables and costs are integer.
- The lower bound comes from the incumbent solution, the upper bound from the nodes of the list; when the list is empty they coincide, and the incumbent solution is certified optimal.
- Each child is re-optimized with the dual simplex algorithm from the dictionary of its parent.
- The selection strategies and a good heuristic solution change the size of the tree, not the optimal value.

Dashboard of the car example. The lower and upper bounds on $z(\text{ILP})$ found so far in this module:

Chapter	Lower bound	Upper bound	From
Integer linear programming	12, 14	–	heuristic solutions $(4, 0)$ and $(4, 1)$
LP relaxation and dual bounds	–	15	$\lfloor z(\text{LP}) \rfloor$, with $z(\text{LP}) = \frac{31}{2}$
Cutting plane algorithm	14	14	13 Gomory fractional cuts
This chapter, execution A	14	14	6 LP problems solved
This chapter, execution B	14	14	8 LP problems solved
This chapter, with LB = 14	14	14	3 LP problems solved

In the car example the algorithm closes the gap between LB = 13 (rounded LP solution) and UB = 15 (LP relaxation) by solving 6 LP problems.